

Engineering Brief

Atrium > Metacron > Pathfinder

Three products on a shared substrate: different audiences, different surfaces, same backend.

Atrium

THE INTERNAL COCKPIT

What it is

The internal cockpit for Unicon operators. Where Kyle, Keenan, and Curtis see the company breathe, direct agents, and edit the system

How it works

Atrium is a feature-flagged, tenant-scoped surface inside the unicon-platform Vite app. Auth is Supabase magic link plus a hardcoded email allowlist (kyle, keenan, curtis, team @unicon.systems). Eight tabs: Now, People, Work, Money, Marketing, Products, System, Library. Reads from the shared substrate; edits route through the refusal layer (Taboo Keeper) before they persist.

Who it's for

Internal company. No customers see Atrium.

Stack

- Vite + React 19 + TypeScript + Tailwind
- Supabase (auth, RLS, Realtime, pgvector)
- Recharts for data viz
- Slack Orchestrator app (custom Slack workspace app, Bolt-style event handling)
- Inngest for durable agent workflows
- Vercel hosting (auto-deploy from main)

Current status

- Sprints 0, 1, 2 verified. Sprint 3 deployed, awaiting Verified.
- Live: Atrium shell with auth, Now tab with skills surface stub, Slack Orchestrator running Sonnet 4.6, agent runtime, Analyst nightly cron, Elder and Taboo Keeper registered, ingest pipeline writing call records to a unified ledger.
- In flight: Sprints 4 through 7 queued under a Master Conductor that runs them autonomously.

Prototype-hardcoded vs production-ready

Layer	State
Auth + email allowlist	Production-ready (allowlist hardcoded; Google OAuth queued for Sprint 7)
Ingest pipeline (calls, Slack, email, mobile)	Production-ready architecturally; voice memo and Apple Notes in Sprint 4; email cron in Sprint 5
Slack Orchestrator	Production-ready as a tool surface; tool set grows as new specialists register
Now tab — skills surface	Stub; populated in Sprints 3-6 as each domain's skills land
Now tab — status pulse, activity feed, quick capture	Designed in Claude Design batch; Sprint 4 wires
Work, People, Money, Marketing, Products, Library tabs	Mostly designed; partially built; Sprints 4-6 fill
System tab (agents galaxy, taboos, refusal log, services, decay, scheduled jobs, audit log)	Sprint 3 built the core, Sprint 7 polishes
PWA install on phone	Sprint 7
Production observability (Sentry, alerting, dashboards)	Not started; gap to fill
Performance (chart bundle size, lazy-load tabs)	Sprint 7 polish; needs production-grade audit
Customer/financial/marketing data	Mostly placeholder; needs real connectors (Stripe, accounts, etc.)
Mobile parity	Sprint 4 (Now + Work) and Sprint 7 (rest)

Specs coverage

- Five SPECs in **Company Docs/Specs/** (parent + four addenda) define every layer of the architecture, from schema to UI to refusal gate to closed-loop feedback to scenario-satisfaction validation.
- Seven per-sprint prompts in **Company Docs/Prompts/** get us to a feature-complete v1 by end of Sprint 7. The Master Conductor runs them sequentially without human intervention except critical halts.
- After Sprint 7: Atrium has all 8 tabs functional, mobile-responsive on Now and Work, PWA installable, refusal layer fully wired to every editable surface, design tokens consistent throughout.

What's NOT in the SPECs (engineer's domain to harden)

- Production observability (Sentry, structured logging, alerting on agent budget breach, alerting on refusal-layer failure)

- Performance auditing (Lighthouse target above 85 is in Sprint 7 but real-world tuning lives with the engineer)
- Real connectors for Money tab (Stripe, banking, accounting) replacing placeholder data
- Production-grade error boundaries, retry logic, fallback UI when MCP tools time out
- Hardening the email allowlist into a real auth model when team expands beyond four
- Offline behavior for the PWA beyond the basic service worker

Metacron

THE OPERATOR CONSOLE

What it is

The operator console for creating and managing customer agent fleets. Where team admins configure agents, review Architect proposals, monitor agent runs.

How it works

Same codebase as Atrium (**unicron-platform**) with multi-tenant scope. Different host or route serves the Metacron view to non-Atrium-allowlisted users. Customer admin authenticates, sees their tenant's agent fleet, can edit configs that route through the same Taboo Keeper validation.

Who it's for

Customer admins (Zedcor admin today, future tenants tomorrow). Also Unicon operators when working across tenants.

Stack

- Vite + React 19 + TypeScript + Tailwind (sibling of Atrium)
- Supabase RLS for tenant isolation
- Multi-tenant routing via host detection (atrium.unicron.systems → Atrium; default host → Metacron)
- Same Inngest + LLM gateway as the agent backend

Current status

- Backend agents are running in production for Zedcor: Architect (decomposition, weekly tuning, weekly discovery), Source Onboarder (Tier 1 sources), Coverage Expansion, Tier 2 human-assist queue.
- Frontend operator surface exists at unicon-platform.vercel.app. Functional for our internal use; rough for non-Unicon customer admin self-service.
- Voice Builder UI exists at unicon-voice-prototype.vercel.app/builder; will be absorbed into Atrium → System → Voice in Sprint 6 per Addendum 3.

Prototype-hardcoded vs production-ready

Layer	State
Backend agent fleet (Architect, Source Onboarder, Coverage Expansion)	Production-ready; running for Zedcor
Multi-tenant schema (RLS, customer_id scoping)	Production-ready
Operator console UI	Functional but rough; needs polish for non-Unicon operator use

Customer admin onboarding flow	Not built; today we hand-onboard each customer
Per-tenant configuration UI for non-engineers	Partial; procurement_pull configs editable in /builder, otherwise mostly database edits
Billing integration (Stripe write-through)	Not started
Per-tenant usage and cost visibility	Hooks exist in nervous_system.connected_services; UI not built
SSO for customer admins	Not started; Sprint 7 covers SSO at the platform level
Customer-facing documentation	Not started

Specs coverage

- The Atrium SPEC and Addenda define operator UI primitives that Metacron will reuse (galaxy view, agents tab, taboos editor, refusal log, services health, scheduled jobs).
- Pathfinder Phase 2 SPEC (separate document at **Company Docs/PRD/PRD - Phase 2 Tailored Pathfinder.md**) covers multi-tenant routing and tenant config layer.
- Voice integration into Metacron + Atrium covered by Addendum 3.

What's NOT in the SPECS (engineer's domain to harden)

- Customer admin onboarding flow (self-service signup → tenant provisioning → agent configuration → first call placed → outcome verified)
- Billing integration (Stripe customer creation, subscription management, usage-based metering for voice minutes and LLM tokens)
- Production observability scoped per tenant (which customer's agents are healthy, who is over budget, who has refusal-layer hits)
- Customer-facing docs and onboarding guides
- SSO integration for customer admins (Google, Microsoft, custom SAML for enterprise)
- Compliance and audit reports per tenant (SOC 2 friendly logging, data export, deletion)

Pathfinder

THE CUSTOMER-FACING APP

What it is

The customer-facing lead intelligence app. End-users (sales agents, BD reps) see ranked leads, draft outreach, run cross-pollination across their network. Voice agents call procurement offices and write structured records as new leads.

How it works

Next.js 14 app, proxied. Per-tenant data scoped via `customer_id` foreign keys. Nine specialist agents pull leads from public sources, rank, verify, enrich, score by adjacency to existing customer relationships, draft outreach, brief the sales agent. Voice agents add a fifth ingest source (procurement offices without APIs).

Who it's for

Customer end-users (Zedcor's BD reps now). Future: any tenant's sales team.

Stack

- Next.js 14 + TypeScript + Tailwind
- Supabase (pathfinder schema, RLS, pgvector for semantic search across leads)
- Inngest for durable workflows (ingest cron, agent dispatch)
- Vercel hosting + cron
- Perplexity API for enrichment
- Vapi + ElevenLabs + Deepgram for voice agents
- HubSpot connector for SDR write-through (in progress)
- Notion + Slack connectors for operator surface

Current status

- Live for Zedcor pilot. Customer-zero is on the system.
- 9 Pathfinder specialists running: Ingestor (sam.gov, USAspending, Harris County, news), Ranker, Verifier, Enricher (via Perplexity), AdjacencyMapper, GeoMapper, Outreach Drafter, Briefer, Cross-Pollinator.
- Cross-pollination engine shipped.
- Voice agents (Discovery, SDR, Procurement Pull) working at `unicron-voice-prototype.vercel.app`; writing to `pathfinder.projects` with `source='voice_agent'`.
- 601 leads in the system as of recent count.

Prototype-hardcoded vs production-ready

Layer	State
Public-data ingest (sam.gov, USAspending, Harris County, news)	Production-ready; running on cron
Voice ingest (procurement_pull)	Production-ready architecturally; Plaud deferred (no public API), Fathom is primary
Lead scoring + ranking	Working; needs ongoing tuning
Adjacency + cross-pollination	Production-ready; engine shipped
Customer-end-user UI	Functional for Zedcor; needs polish for new customer onboarding
Multi-tenant onboarding	Partial; schema is multi-tenant but UI flow assumes hand-onboarding
HubSpot write-through (SDR loop)	In progress, not yet shipped
Voice agent self-service config	Hand-configured today; Atrium → System → Voice (Sprint 6) puts it in operator UI
Production observability	Partial; agent run logs in audit_log, but no alerting or dashboards
Mobile-responsive customer UI	Functional but not optimized
Lead quality validation	Boolean checks today; satisfaction-gated per Addendum 4 is queued

Specs coverage

- Pathfinder Phase 2 PRD covers multi-tenant scope.
- Addendum 3 covers voice integration into the operator surface.
- Addendum 4 covers scenario-driven satisfaction validation (replaces brittle boolean tests for agent-built outputs).
- Sprints 1-7 do NOT directly ship Pathfinder customer-facing features. Pathfinder gets the ingest pipeline upgrade in Sprint 1, voice agent registration in Sprint 5, and customer activity views in Sprint 5/6 of Atrium. Pathfinder's own roadmap runs in parallel and is owned by separate Cowork chats.

What's NOT in the SPECS (engineer's domain to harden)

- Multi-tenant customer onboarding flow (signup → contract → tenant provision → first lead delivered)
- Self-service voice agent configuration (today configs are hand-edited in pathfinder.procurement_pull_configs)
- Mobile-responsive customer UI (Pathfinder's own surface, separate from Atrium mobile)

- Production observability scoped per customer (lead delivery rate, ingest health, agent budget, refusal-layer hits)
- Lead quality satisfaction validation per Addendum 4 (replacing boolean tests with LLM-judge satisfaction gates)
- HubSpot write-through completion (SDR call outcomes → HubSpot deals)
- Customer-facing analytics: per-customer lead delivery dashboards, ROI tracking, attribution from lead → closed deal

What this means dev

THE PRODUCTION-HARDENING GAP

The architecture and the agent fleet are real. The closed-loop substrate is real. What we have is not a tech demo. We have customer-zero on it.

What we do not have is the production hardening that takes us from "running for one tenant we hand-hold" to "any new customer can onboard themselves and the system stays healthy without us watching."

That gap is roughly:

- Production observability and alerting across all three products
- Multi-tenant onboarding flows for both Metacron (customer admins) and Pathfinder (customer end-users)
- Real connectors replacing placeholder data on Atrium's Money, Marketing, and Products tabs
- Self-service customer configuration (no hand-onboarding)
- Billing integration
- Compliance and audit reporting per tenant
- Performance auditing and optimization beyond the Sprint 7 polish pass
- Lead quality satisfaction validation per Addendum 4 once the substrate ships

The five SPECs and seven sprint prompts get us through the architecture and the cockpit. They do not cover the production-hardening work above. **That is where an engineer's judgment lives.**

Read the SPECs in **Company Docs/Specs/** and the per-sprint prompts in **Company Docs/Prompts/** for full context. The handoff brief at **Company Docs/Context/HANDOFF - Internal Org Cowork Chat.md** summarizes everything in one read.

Questions welcome. Push back on anything that smells off.

Unicron Systems · Internal · 2026-05-08